

BUPT SE ICP Quiz -- Unit 07 Records and Structure

If you are using LLM to help you in this quiz, you MUST write down 1) the name of the LLM and 2) the PROMPTS. Otherwise, you will get zero score if the instructor finds that you are using LLM.
You may use the following table as a template, putting the table down to the title of each question.

Name of LLM	Prompts
-------------	---------

- [T] True or false: The types of each element in an array must be the same.
- [F] True or false: The types of each field in a record must be different.
- [T] True or false: Fields of a record are lvalues in C.
- [T] True or false: A record is itself an lvalue in C.
- Define a student structure that contains four member variables: name, student ID, age, and grade, with data types of string, integer, integer, and floating-point, respectively. You do NOT need to write a main() function.

```
struct Student {  
    char name[20];  
    int StudentID;  
    int age;  
    float grade;  
}
```

- Declare a student structure variable and input the information of the student from the keyboard, Then output the information of the student.

```
#include <stdio.h>
```

```
typedef struct{  
    int studentID;  
    int age;  
    char name[20];  
    float grade;  
}Student;
```

```
int main(){  
    Student student;  
    printf("plz type the student's name");  
    scanf("%s", student.name);  
    printf("plz type his studentID");  
    scanf("%d", &student.studentID);  
    printf("plz type his age");  
    scanf("%d", &student.age);  
    printf("plz enter his grade");  
    scanf("%f", &student.grade);  
  
    printf("The student %s, with the student ID %d, aged %d, gets  
%.1f\n", student.name, student.studentID, student.age, student.grade);  
    return 0;  
}
```

7. Define an enumeration type `WeekDay` representing the day of the week, with seven enumeration elements: Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, and Sunday. Test `WeekDay` in `main()`.

```
#include <stdio.h>
```

```
typedef enum {  
    MON,  
    TUE,  
    WED,  
    THU,  
    FRI,  
    SAT,  
    SUN  
} Weekday;  
  
int main(){  
    int today = TUE;  
    switch(today){  
        case MON : case TUE : case WED : case THU : case FRI :  
            printf("Today is WEEKDAY\n");  
            break;  
        case SAT : case SUN :  
            printf("Today is WEEDKEND\n");  
            break;  
    }  
}
```

8. Based on Quiz7, define a date structure that contains four member variables: year, month, day and weekday, with data types of integer, integer, integer, `WeekDay` respectively. Store the current date in a date structure variable and then output the date.

```
#include <stdio.h>
```

```
typedef enum{  
    MON,  
    TUE,  
    WED,  
    THU,  
    FRI,  
    SAT,  
    SUN  
} Weekday;  
  
struct Date {  
    int year;  
    int month;  
    int day;  
    Weekday weekday;  
} ;  
  
int main(){
```

- ```
#include <stdio.h>
```

```
typedef struct{
 char name[20];
 int stuID;
 float grades[3];
} stuGrade;

int main(){
 stuGrade student;
 printf("Plz type the student's name _____\b\b\b\b\b\b\b");
 scanf("%s",student.name);
 printf("Plz type the student ID of the student: ");
 scanf("%d",&student.stuID);
 printf("Plz type the grade of the Student \n");
 printf("The first subject? ");
 scanf("%f",student.grades);
 printf("The next? ");
 scanf("%f",student.grades+1);
 printf("The last one ");
 scanf("%f",student.grades+2);
}
```

- ```
#include <stdio.h>
```

```
typedef struct{  
    char name[20];  
    int stuID;  
    float grades[3];  
} stuGrade;  
  
int main(){  
    stuGrade student;  
    printf("PLz type the student's name _____\b\b\b\b\b\b\b\b\b\b");  
    scanf("%s",student.name);
```

11. Based on Quiz10, define a student grade array that may contain 3 elements. Define a function calcAverage to compute the average score of the student grade array.

```
#include <stdio.h>
```

```
typedef struct{
    char name[20];
    int stuID;
    float grades[3];
} stuGrade;

float calAverage(float *s){
    return (*s+*(s+1)+*(s+2));
}

int main(){
    stuGrade student;
    printf("Plz type the student's name _____\b\b\b\b\b\b\b\b");
    scanf("%s",student.name);
    printf("Plz type the student ID of the student: ");
    scanf("%d",&student.stuID);
    printf("Plz type the grade of the Student \n");
    printf("The first subject? ");
    scanf("%f",student.grades);
    printf("The next? ");
    scanf("%f",student.grades+1);
    printf("The last one ");
    scanf("%f",student.grades+2);

    float total = *(student.grades) + *(student.grades + 1) +
        *(student.grades + 2);
    printf("Total grade is %.1f\n",total);
    printf("The Average grade is %.1f\n",calAverage(student.grades));
}
```

- ## 12. Traffic Light Control System

Objective: To demonstrate your understanding of enums by creating a simple traffic light control system. Problem Statement: A traffic light operates in a sequence of colors: RED, GREEN, and

YELLOW. Create a C program that uses an enum to represent these colors and simulates the traffic light sequence. The program should display each color for a specified duration (e.g., 3 seconds for RED, 5 seconds for GREEN, and 2 seconds for YELLOW). You can use a `sleep()` function to pause the program for the specified durations.

Requirements: Define an *enum* named `TrafficLightColor` with values RED, GREEN, and YELLOW. Create a function *void changeTrafficLight (TrafficLightColor color, int duration)* that prints the current color and pauses for the specified duration. Implement a main function that simulates the complete traffic light cycle.

Sample Output:

Traffic Light is RED for 3 seconds.

Traffic Light is GREEN for 5 seconds.

Traffic Light is YELLOW for 2 seconds.

Hints: Use `#include <windows.h>` for `Sleep()` function in Visual Studio IDE. Alternatively, you can use a loop with a delay to simulate the sleep functionality if `sleep()` is not available.

```
#include <stdio.h>
#include <unistd.h> //内置 sleep()

typedef enum{
    RED,
    YELLOW,
    GREEN
} trafficLightColor;

const char* light[3] = {
    "Red",
    "Yellow",
    "Green"
};

void changeTrafficLight(trafficLightColor color, unsigned int
*duration){
    static int i = 0;
    while (1){
        color = i % 3;
        printf("The traffic light color now is %s\n", light[color]);
        sleep(duration[color]);
        i ++;
    }
}

int main(){
    trafficLightColor color = RED;
    unsigned int duration[3] = {3, 5, 2};
    changeTrafficLight(color, duration);
}
```

13. Student Grade Management System

Objective: To practice using both structs and enums by developing a simple student grade management system.

Problem Statement: Develop a C program that manages grades for a class of students. Each student has a name, a student ID, and an array of grades for various subjects. The grades should be

stored using an enum with values A, B, C, D, and F. The program should allow you to:

- Add a new student with their name, ID, and grades.
- Calculate and display the GPA (Grade Point Average) for each student using the following point system: A=4.0, B=3.0, C=2.0, D=1.0, F=0.0.
- Display the class average GPA.

Requirements:

- Define an enum named Grade with values A, B, C, D, and F.
- Define a struct named Student that includes:
 - ✓ char name[50]
 - ✓ int id
 - ✓ Grade grades[5] (assuming each student has grades for 5 subjects)
- Create functions to:
 - ✓ Add a new student.
 - ✓ Calculate the GPA for a student.
 - ✓ Calculate and display the class average GPA.
- Implement a main function that demonstrates these functionalities.

Sample Input:

```
Enter number of students: 2
Enter name, ID, and grades for student 1:
Name: Alice
ID: 101
Grades: A B C D F
Enter name, ID, and grades for student 2:
Name: Bob
ID: 102
Grades: B B B B A
```

Sample Output:

```
Student Alice's GPA: 2.2
Student Bob's GPA: 3.4
Class Average GPA: 2.8
```

Hints:

- Use struct arrays to store multiple students and their grades.
- Use loops to input grades and calculate GPAs.
- Remember to handle conversions from enum values to their corresponding GPA points.

[This problem is a bit challenging. The solution will require approximately 130 lines of code, so a bit of patience is needed. Embrace the challenge!]

```
#include <stdio.h>
#include <string.h>

#define MAX_STUDENTS 100
#define MAX_NAME_LEN 50

typedef enum {
```

```
    A, B, C, D, F
} Grade;

typedef struct {
    char name[MAX_NAME_LEN];
    int id;
    Grade grades[5];
} Student;

// Function to convert a character grade to enum Grade
Grade char_to_grade(char ch) {
    switch(ch) {
        case 'A': return A;
        case 'B': return B;
        case 'C': return C;
        case 'D': return D;
        case 'F': return F;
    }
}

// Function to map Grade enum to points
float grade_to_points(Grade g) {
    switch(g) {
        case A: return 4.0;
        case B: return 3.0;
        case C: return 2.0;
        case D: return 1.0;
        case F: return 0.0;
    }
}

// Add new student
int add_student(Student students[], int idx) {
    printf("Enter name, ID, and grades for student %d:\n", idx+1);
    printf("Name: ");
    // Remove extra newline from previous input if there
    getchar();
    fgets(students[idx].name, MAX_NAME_LEN, stdin);
    size_t len = strlen(students[idx].name);
    if(students[idx].name[len-1] == '\n')
        students[idx].name[len-1] = '\0';
    printf("ID: ");
    scanf("%d", &students[idx].id);
    printf("Grades: ");
    for (int i=0; i<5; ++i) {
        char grd[3];
        scanf("%2s", grd);
        students[idx].grades[i] = char_to_grade(grd[0]);
    }
    return 1;
}

// Calculate GPA
```

```
float calculate_gpa(const Student* student) {
    float total = 0.0;
    for (int i = 0; i < 5; ++i)
        total += grade_to_points(student->grades[i]);
    return total / 5;
}

//show GPA
void display_class_average(const Student students[], int n) {
    float sum = 0.0;
    for (int i = 0; i < n; ++i)
        sum += calculate_gpa(&students[i]);
    printf("Class Average GPA: %.1f\n", sum/n);
}

int main() {
    int n;
    printf("Enter number of students: ");
    scanf("%d", &n);
    Student students[MAX_STUDENTS];

    for (int i=0; i<n; ++i) {
        add_student(students, i);
    }

    for (int i=0; i<n; ++i) {
        float gpa = calculate_gpa(&students[i]);
        printf("Student %s's GPA: %.1f\n", students[i].name, gpa);
    }

    display_class_average(students, n);

    return 0;
}
```